

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the `getprocs()` system call and test it using the provided `ps` command.

`getprocs()` takes a pointer to a user-space array of `struct pstat`. The kernel uses `copyout()` to copy the `struct pstat` (`pid`, `status`, `memory size`, `parent pid`, `name`) to the caller's array for each process in the process table whose state is not `UNUSED`. The number of processes discovered is returned. In compliance with `xv6`'s lock ordering, `wait_lock` is held while reading `p->parent`, and each `p->lock` is held while that process's fields are read.

- `kernel/pstat.h` - Added `struct pstat`. Moved enum `procstate` here from `proc.h` so user programs like `ps` can use the state names.
- `kernel/proc.h` - Removed the `procstate` enum and included `pstat.h`
- `user/user.h` - Added `struct pstat`; and the prototype `int getprocs(struct pstat*)`;
- `user/usys.pl` - Added entry("`getprocs`"); so the user-side stub is generated.
- `kernel/syscall.h` - Added `#define SYS_getprocs <N>`
- `kernel/syscall.c` - Added the extern declaration and the `syscalls[]` table entry for `sys_getprocs`.
- `kernel/defs.h` - Added the prototype for `kgetprocs()`.
- `kernel/sysproc.c` - Implemented `sys_getprocs()`, which reads the pointer argument with `argaddr()` and calls `kgetprocs()`.
- `kernel/proc.c` - Implemented `kgetprocs()`, which does the table walk and `copyout()`.
- `user/ps.c`, `Makefile` - Added the provided `ps` to `user/` and to `UPROGS`. Added the `kernel/param.h` and `kernel/pstat.h` includes to `ps.c`.

```
pid    state    size    ppid    prio    age    name
1      sleeping  16384   0        10     -     init
2      sleeping  20480   1        10     -     sh
3      running   16384   2        10     -     ps
$
```

Task 2. (15 pts) Modify the provided `ps` command to print the priority of each process.

Changes Made:

- `kernel/proc.h` - Added `int priority`; to `struct proc`.
- `kernel/proc.c` - `allocproc()` initializes `p->priority` to the default (10). `fork()` copies the parent's priority to the child. `kgetprocs()` copies `p->priority` into the `pstat` entry.
- `kernel/pstat.h` - Added `int priority`; to `struct pstat`.
- `user/ps.c` - Added a `prio` column to the header.

- kernel/sysproc.c, kernel/syscall.h, kernel/syscall.c, user/user.h, user/usys.pl
 - Added the getpriority() and setpriority() system calls (syscall number, table entry, handler, prototype, user stub).

```
$ prio
initial priority: 10
after setpriority(3): 3
child inherited priority: 3
pid    state    size    ppid    prio    age    name
1      sleeping  16384   0       10     -     init
2      sleeping  20480   1       10     -     sh
4      sleeping  16384   2       3      -     prio
6      running   16384   4       3      -     ps
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

ps calls uptime() once to get the current tick count, and for each RUNNABLE process prints age = now – readytime, which is the number of ticks that process has been waiting since it last became runnable. For processes that are not RUNNABLE it prints -.

```
pid    state    size    ppid    prio    age    name
1      sleeping  16384   0       10     -     init
2      sleeping  20480   1       10     -     sh
13     running   16384   2       10     -     ps
17     runnable  16384   1       10     4     matmul
19     runnable  16384   1       10     3     matmul
21     runnable  16384   1       10     2     matmul
23     runnable  16384   1       10     1     matmul
25     runnable  16384   1       10     0     matmul
```

Task 4. (15 pts) Implement a priority-based scheduler.

- kernel/param.h
 - SCHED_RR, SCHED_PRIORITY, SCHEDPOLICY
 - Compile-time constants that select the scheduling policy.
- kernel/proc.c
 - scheduler()
 - Under #if SCHEDPOLICY == SCHED_PRIORITY, scans the whole process table, keeps the lock of the highest-priority RUNNABLE process, releases the others, and runs that process. Ties go to the first one found. The original round-robin loop is kept under #else.
- kernel/proc.h, kernel/proc.c, syscalls
 - priority field, setpriority()/getpriority()
 - Carried over from Task 2; needed so programs can set the priority the scheduler uses.

I learned from implementing the priority scheduler that the scheduler is just a loop that determines which RUNNABLE process gets the CPU next, and that changing the policy only affects how that decision is made. Additionally, I learned how locking works around a context switch. The selected process's lock must remain preserved while the others are released because the scheduler gets a process's lock before `switch()` and the process releases it. The test results showed a limitation of strict priority scheduling because a low priority process may starve since it only executes once each higher priority process is finished.

```
$ schedtest
child with priority 20 finished at tick 33
child with priority 10 finished at tick 37
child with priority 5 finished at tick 40
$ schedtest
child with priority 10 finished at tick 43
child with priority 10 finished at tick 47
child with priority 10 finished at tick 51
```

Task 5. (15 pts) Add aging to your priority-based scheduler.

Each process has a boost that is added to its base priority to give an effective priority. For every `AGING_INTERVAL` (10) ticks a process spends waiting in the RUNNABLE state, its boost increases by 1. For every `AGING_INTERVAL` ticks it spends RUNNING, its boost decreases by 1 but never below 0. The scheduler picks the RUNNABLE process with the highest effective priority. The base priority is never changed, so `getpriority()` and `ps` still show what the user set.

AgeTest set to on:

pid	state	size	ppid	prio	age	name
1	sleeping	16384	0	10	-	init
2	sleeping	20480	1	10	-	sh
6	zombie	16384	5	5	-	agetest
5	sleeping	16384	1	30	-	agetest
7	zombie	16384	5	1	-	agetest
8	zombie	16384	5	5	-	agetest
9	zombie	16384	5	5	-	agetest
10	running	16384	2	10	-	ps

```
$ LOW (prio 1): response 7, turnaround 11
average response 0, average turnaround 4 (12 procs)
```

AgeTest set to off:

```
$ agetest & ps
pid      state      size    ppid    prio    age    name
1        sleeping   16384    0       10     -     init
2        sleeping   20480    1       10     -     sh
3        running    16384    2       10     -     ps
5        runnable   86016    1       10     0     sh
$ LOW (prio 1): response 324, turnaround 355
average response 105, average turnaround 135 (12 procs)
```

What I learned:

I discovered that aging fixes starvation by allowing a process's effective priority to increase the longer it waits. Strict priority scheduling causes starvation. I also discovered that since all waiting processes age at the same pace and the difference between older ones never goes down, aging only serves to boost a waiting process against more recent arrivals. My test required a steady flow of new, high-priority workloads because of this. The results showed a trade-off that while aging reduced the response and turnaround time of low-priority processes, it also caused some high-priority activities to be delayed, meaning that the average turnaround did not significantly improve. Instead of a reduced average turnaround, aging provides fairness and a limit on waiting time.

Task 6. (5 pts) Summary

This project showed me how a kernel decides what runs next and exposes process information to user programs. I learned through `getprocs()` that the scheduler was a loop where changing the policy meant changing one selection step, and that the kernel must `copyout()` each `struct pstat` into the caller's memory instead of returning it directly. Additionally, I observed that aging addresses the issue of low-priority processes being starved by rigid priority scheduling, but at the cost of some high-priority operations being delayed. One problem was that `SLEEPING` and `RUNNABLE` are specified in kernel headers, which prevented `ps.c` from compiling. I resolved this by shifting enum `procstate` into `pstat.h`. Another was an "unknown sys call 25" error, caused by a missing entry in the syscall dispatch table, which I fixed by checking every file a new syscall touches.

Extra Credit Task (10 points).

Resources:

Anthropic Claude: Claude AI was used for debugging and assistance with understanding of processes and functions within the given files. Examples of some prompts used were “Analyze the following code and provide suggestions and feedback on any possible or current issues” or “Explain the following code and how it coexists and functions with the other following files or methods.” Suggestions used by Claude were considered and used for the completion of the project when an issue resulting in a substantial amount of time being used to resolve it.